

Error Handling

A compiler should:

- detect errors
- locate errors
- recover from errors

Errors may occur in any of the three main analysis phases of compiling:

- lexical analysis
- syntax analysis
- semantic analysis

Lexical Errors

few possibilities:

- unexpected character
- illegal sequence of characters

recovery strategy:

- delete offending character from input and continue

Syntax Errors

Most errors in programs can be detected in this phase.

The parser should:

- detect all syntactic errors
- report errors clearly and accurately
- recover quickly and detect subsequent errors
- not degrade compiler efficiency

The main difficulty in designing the error handling capabilities of a compiler is balancing these competing objectives.

Error Recovery

Poor error recovery causes an avalanche of error messages, which are not errors in the source program.

There are a number of different strategies:

panic mode - ignore all symbols until a "synchronising" token is found
e.g. and **"end"** or **";"**, etc.

- simple to implement
- guaranteed to halt
- ignores a lot of code

phrase level - replace a prefix of current input by string allowing parser to continue.
Normally replaces/deletes delimiters.

- danger of looping
- unable to deal with cases where error is on stack and not on input

Error Recovery (cont.)

error productions - include extra productions in grammar which recognise commonly occurring errors

- requires analysis of language use
- ensures messages and recovery procedures are specific to the actual error

global correction - compiler carries out minimum number of changes to get a correct program

- algorithms exist to determine minimum change
- requires complete traversal of program before correction
- extremely expensive in time and space

Error Handling in LR Parsing

Detection:

detects errors at earliest possible point in input, by reaching error entry in parse table.

Recovery:

panic mode - look down stack until state s which has a table entry of **Si** for one (A) of a set of particular non-terminals. Throw away input symbols until one from $\text{follow}(A)$ is reached.

Discard states above s , place i on stack.

Assumes error was found in derivation from A, and pretends instead that an A was parsed.

For example, if A is a nonterminal *stmt*, *follow(stmt)* contains ‘.’

phrase level - for each error entry in table, insert a pointer to a particular error procedure, which assumes the most likely cause, and takes the appropriate action.

	S		T		F
0			<i>a</i>		+
1			*		(
2				)	
3		\$			
4	1		2		3
5			S5	e1	e1
6		S4		e2	e1
7					
8			e3	S6	e4
9		e3		e2	A
10					
11			e3	R2	S7
		e3		R2	R2

Error Procedures

- e1:** /* expecting an **a** or an "(", but finding an "+", "*" or a "#" */
 put 5 on top of stack /* assume **a** */
 issue "missing operand" message
- e2:** /* finding an unexpected ")" */
 remove ")" from input /* ignore it */
 issue "unmatched right parenthesis" message
- e3:** /* expecting "+", finding **a** or "(" */
 put 6 on top of stack /* assume "+" */
 issue "missing +" message
- e4:** /* expecting "+", finding "*" */
 put 6 on top of stack /* assume "+" */
 remove "*" from input
 issue "* instead of +" message
- e5:** /* expecting ")", finding "#" */
 put 11 on stack /* assume ")" */
 issue "missing right parenthesis" message

Yacc Error Recovery

Yacc uses error productions (but similar to the phrase-level recovery given above).

Add productions $A \rightarrow \text{error } \alpha$

When error is found, scan down stack until a state with a member $A \rightarrow \cdot \text{error } \alpha$ is found.

Scan through input until a string matching α is found, and remove all input until the end of that string.

Parser then reduces to A , and continues.

Example:

$\text{Stmt} \rightarrow \text{error} ;$